

SENG 696 Report 1: University Student Registration Agents

Ayah Metwali, 30042982

This report will be in the order below:

1) System Specifications (Pg. 2)

- Business Case, pg. 2
- System Description, pg. 2
- Assumptions, pg. 3
- Requirements, pg. 4
- Wish List (Not Implemented) pg. 4

2) Design Document (GAIA Methodology) (Pg 5)

- Requirements Statement pg. 5
- Goal Identification pg. 5
- Analysis Phase pg. 6
 - o Role Model pg. 6
 - o Interaction Model pg. 10
- Design Phase pg. 11
 - o Agent Model pg. 11
 - o Service Model pg. 11
 - o Acquaintance Model pg. 12
- Agent Description pg. 13
- Agent Internal Architecture pg. 15
- Extensions and Limitations (Preliminary) pg. 15

1) System Specifications:

- ***Business Case***

University course registration is a recurring process that competes on two constrained resources at once: seats in individual sections and the finite time slots in a student's own schedule. Students must follow degree requirements, prerequisite chains, and section availability across multiple departments, often within a narrow registration window.

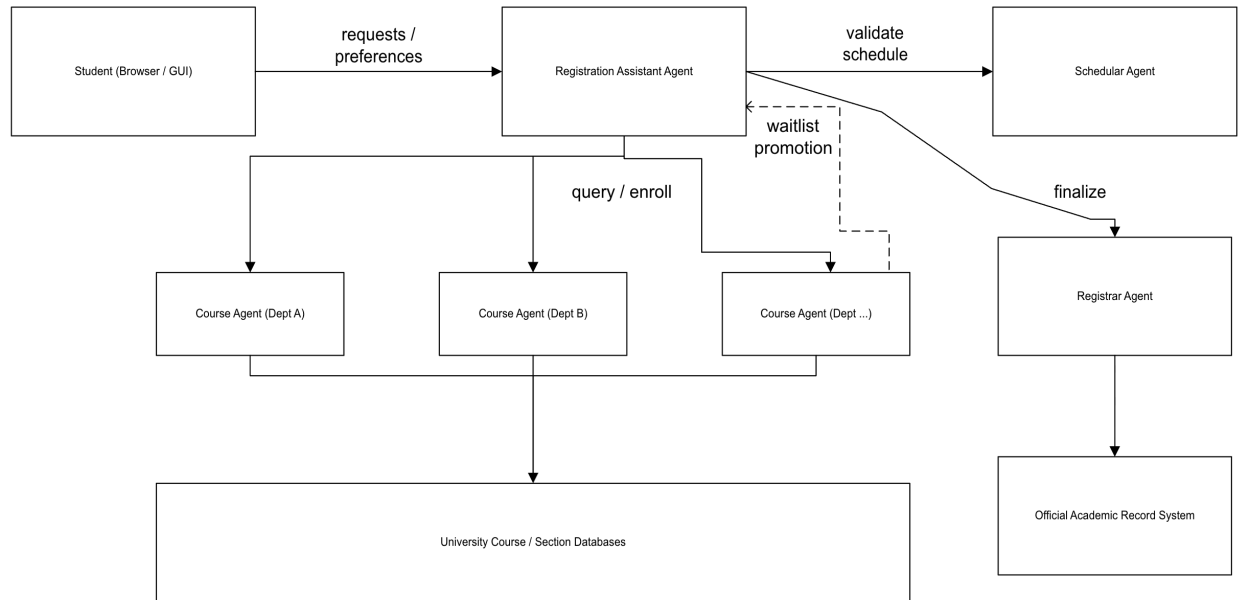
Existing registration systems act as record keepers where they display seat counts and let a student attempt an enrollment, but the burden of checking for time conflicts, verifying prerequisites, and monitoring waitlists across several departments falls on the student. As course catalogues grow and popular sections fill within minutes of opening, this manual coordination becomes error-prone and time-consuming.

A multi-agent approach can assist in this problem because the underlying task is distributed where each department controls its own section data and enrollment rules, while the students' overall schedule is separate. Modelling departmental course data and the scheduling logic as autonomous, cooperating agents allows each concern to be owned by the agent best positioned to reason about it, while a coordinating agent assembles the results into one recommendation for the student.

- ***System Description***

We will be calling this system as the university course registration and scheduling assistant (UCRSA). It is a multi-agent system that helps a student assemble a conflict-free, prerequisite satisfying course schedule by coordinating a Registration Assistant Agent, a Scheduler Agent, a set of department level Course Agents, and a Registrar Agent.

The student interacts with a single Registration Assistant Agent through a browser-based interface. This agent collects the student's course preferences, consults the Scheduler Agent to check the proposed selection against the student's existing schedule and degree requirements, queries the relevant Course Agents for section availability, and once the student approves a schedule, submits it to the Registrar Agent to be finalized.



- **Assumptions**

- Each student has an existing profile that records their declared program, completed course history, and degree requirements.
- The student specifies desired courses, acceptable time windows, and a maximum course load through the browser interface. Recurring preferences (e.g., no classes before 9:00 AM) may also be stored in the student profile in advance.
- Course, prerequisite, and seat-availability data are already exposed to UCRSA through a standardized interface agreed by all departments (as in every Course Agent shares a common message schema for course codes, prerequisites, and seat counts).
- Enrollment and waitlist transactions are small at the section level. A single request either succeeds, is queued on a waitlist, or fails outright.
- A student's schedule may be finalized with some courses enrolled and others waitlisted. UCRSA does not require every requested course to succeed before registration and proceed.
- The Registrar Agent holds a student's official registration record.

- **Requirements**

- UCRSA will:
 - Provide course and section inquiries, including seat availability, meeting times, and prerequisites, across multiple departments.
 - Query multiple department Course Agents to source candidate sections for a requested course.
 - Verify a proposed schedule against the student's existing enrollments to detect time conflicts before any enrollment request is submitted.
 - Verify that prerequisite requirements are satisfied before allowing an enrollment request to proceed.
 - Enroll the student in a section or place the student on that section's waitlist if no seats remain, up to the student's approval.
 - Notify the student of any change to their registration status, including a move off a waitlist or a conflict discovered after an initial request.
 - Monitor current waitlist requests and attempt enrollment automatically when a seat becomes available.

- **Wish List (Not Implemented)**

- Allow the student to run "what if" schedule simulations without committing to a registration transaction.
- Support negotiated section swaps directly between two students.
- Detect an alert the student to list-minute section changes such as a room, time, or instructor change.
- Allow Course Agents to register themselves dynamically with UCRSA through a discovery service as departments add new offerings (rather than being statically made).

2) Design Document

This design will follow the GAIA methodology.

- **Requirements Statement** (repeated from the first section, stated again below):
 - Provide course and section inquiries including seat availability, meeting times, and prerequisites, across multiple departments.
 - Query multiple department Course Agents to source candidate sections for a requested course.
 - Verify a proposed schedule against the student's existing enrollments to detect time conflicts before any enrollment request is submitted.
 - Verify that prerequisite requirements are satisfied before allowing an enrollment request to proceed.
 - Enroll the student in a section or place the student on that section's waitlist if no seats remain, up to student's approval.
 - Notify the student of any change to their registration status, including a move off a waitlist or a conflict discovered after an initial request.
 - Monitor standing waitlist requests and attempt enrollment automatically when a seat becomes available.

- **Goal identification**

The goal of UCRSA is a schedule that is conflict free, prerequisite satisfying, and preference aligned with a waitlist position counting as a valid partial outcome where a seat isn't yet available. All working simultaneously.

With sub goals being:

- Identify candidate sections that satisfy the student's requested courses.
- Verify that prerequisites for each candidate section are met.
- Detect and resolve time conflicts between candidate sections and the student's existing schedule.
- Secure a seat or a waitlist position, for each approved section.
- Finalize the registration transaction and record it in the official academic record.
- Notify the student of any subsequent change in registration status.

- **Analysis Phase**

Role Model: Each agent will have hard and soft roles. The table below summarizes the allocation.

Agent	Hard Role(s)	Soft Role(s)
RegistrationAssistant Agent	PersonalAssistant	PreferenceAdvisor
SchedulerAgent	Scheduler	WorkloadAdvisor
CourseAgent (× one per department)	CourseProvider, WaitlistMonitor	DemandForecaster
RegistrarAgent	Registrar	ComplianceAuditor

The following are the schemas for the roles underneath each agent:

Role Schema		PersonalAssistant (hard)
Description		Acts on behalf of the student. Collects the student's course preferences, coordinates with the Scheduler and Course roles to assemble a candidate schedule, presents it to the student, and submits the approved schedule to the Registrar role.
Protocols / Activities		RequestSchedule, CollectSections, ProposeSchedule, ConfirmRegistration
Permissions		reads StudentProfile; reads DegreeRequirements; generates ProposedSchedule
Responsibilities:	Liveness	PERSONALASSISTANT = (RequestSchedule . CollectSections . ProposeSchedule . ConfirmRegistration) - repeats
	Safety	A proposed schedule is never submitted to the Registrar role unless it has first been reported conflict-free by the Scheduler role.

Role Schema		PreferenceAdvisor (soft)
Description		Ranks the set of valid candidate schedules already cleared by the Scheduler role, according to the student's stated preferences (e.g., avoid early mornings, minimize gaps). Does not affect whether a schedule is valid, only the order in which valid schedules are presented.
Protocols / Activities		RankCandidates
Permissions		reads ValidatedScheduleSet; reads StudentPreferences; generates RankedScheduleList
Responsibilities:	Liveness	PREFERENCEADVISOR = (RankCandidates) - repeats

	Safety	A schedule is never added to the ranked list unless it already appears in the validated candidate set produced by the Scheduler role.
--	--------	---

Role Schema		Scheduler (hard)
Description		Validates a proposed set of course sections against the student's existing schedule and degree requirements. Detects time conflicts and unmet prerequisites before an enrollment request is allowed to proceed.
Protocols / Activities		CheckConflicts, CheckPrerequisites, ReportValidation
Permissions		reads StudentSchedule; reads StudentTranscript; reads DegreeRequirements
Responsibilities:	Liveness	SCHEDULER = (CheckConflicts . CheckPrerequisites . ReportValidation) - repeats
	Safety	A validation report is never marked "clear" while an unresolved time conflict or an unmet prerequisite remains in the proposed set.

Role Schema		WorkloadAdvisor (soft)
Description		Reviews an already-validated schedule's total course load and flags an unusually heavy combination (e.g., several historically high-workload courses in the same term), without blocking registration.
Protocols / Activities		AssessWorkload
Permissions		reads ValidatedSchedule; reads CourseWorkloadRatings; generates WorkloadAdvisory
Responsibilities:	Liveness	WORKLOADADVISOR = (AssessWorkload) - repeats
	Safety	A workload advisory is never generated for a schedule that the Scheduler role has not already validated as conflict-free.

Role Schema		CourseProvider (hard)
Description		Owns a single department's course and section data. Reports seat availability and prerequisite metadata, and processes enrollment and waitlist requests for its own sections. One instance exists per department.
Protocols / Activities		QuerySections, RequestEnrollment, RequestWaitlist, ReportSeatChange
Permissions		reads CourseCatalog; changes SeatCount; changes WaitlistQueue
Responsibilities:	Liveness	COURSEPROVIDER = (QuerySections . (RequestEnrollment or RequestWaitlist)) - repeats

	Safety	SeatCount is never decremented below zero; a student already enrolled in a section is never added to that same section's waitlist.
--	--------	--

Role Schema		WaitlistMonitor (hard)
Description		Periodically checks each section's waitlist against freed seats and promotes the next eligible student, notifying the PersonalAssistant role of the promotion. Unlike CourseProvider, this role runs proactively rather than in response to a single incoming request.
Protocols / Activities		CheckSeatAvailability, PromoteFromWaitlist
Permissions		reads WaitlistQueue; changes SeatCount; changes WaitlistQueue
Responsibilities:	Liveness	WAITLISTMONITOR = (CheckSeatAvailability . PromoteFromWaitlist - optional) - repeats
	Safety	A student is never promoted out of waitlist order; a promotion is never issued for a section with no free seat.

Role Schema		DemandForecaster (soft)
Description		Tracks enrollment and waitlist growth for a department's sections over time and flags a section trending toward sustained oversubscription, as input to the department's own decision to open a new section. Produces no student-facing output.
Protocols / Activities		TrackDemandTrend
Permissions		reads EnrollmentHistory; reads WaitlistQueue; generates DemandAlert
Responsibilities:	Liveness	DEMANDFORECASTER = (TrackDemandTrend) - repeats
	Safety	A demand alert never modifies SeatCount or WaitlistQueue directly; it is advisory only.

Role Schema		Registrar (hard)
Description		Holds the authoritative academic record. Finalizes registration transactions that have already been approved by the student and cleared by the Scheduler role, and issues official confirmation.
Protocols / Activities		FinalizeRegistration, IssueConfirmation
Permissions		changes OfficialRegistrationRecord
Responsibilities:	Liveness	REGISTRAR = (FinalizeRegistration . IssueConfirmation) - repeats

	Safety	A registration is never finalized for a section that the Scheduler role has not reported as conflict-free and prerequisite-satisfying.
--	--------	--

Role Schema		ComplianceAuditor (soft)
Description		Logs every finalized registration transaction against university policy checks (e.g., credit load limits) for audit purposes, without altering the transaction itself.
Protocols / Activities		LogRegistrationEvent
Permissions		reads OfficialRegistrationRecord; generates AuditLogEntry
Responsibilities:	Liveness	COMPLIANCEAUDITOR = (LogRegistrationEvent) - repeats
	Safety	An audit log entry is never created before the Registrar role has finalized the corresponding transaction; an audit log entry never modifies OfficialRegistrationRecord.

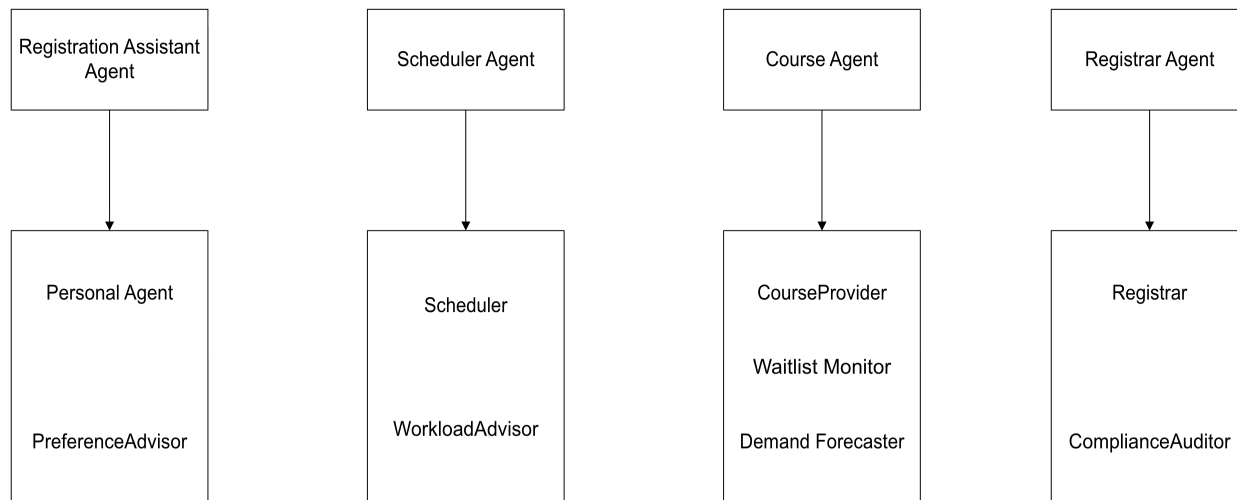
- Interaction Model

Soft roles that only use data already held by their own agent (PreferenceAdvisor, WorkloadAdvisor, DemandForecaster, ComplianceAuditor) do not introduce new inter-agent protocols; WaitlistMonitor is the exception, since a promotion must be reported back to the student.

Protocol	Initiator	Responder	Inputs	Outputs	Processing
RequestSchedule	Student	PersonalAssistant	Desired courses, time preferences	Acknowledgement	Store preferences for the session
CheckConflicts	PersonalAssistant	Scheduler	Candidate section list, existing schedule	Conflict report	Compare section meeting times pairwise
CheckPrerequisites	PersonalAssistant	Scheduler	Candidate section list, transcript	Prerequisite report	Compare completed courses against requirements
QuerySections	PersonalAssistant	CourseProvider	Course code	Section list with seat counts	Filter by term and status
RequestEnrollment	PersonalAssistant	CourseProvider	Student ID, section ID	Enrollment confirmation or waitlist position	Decrement seat count or queue request
FinalizeRegistration	PersonalAssistant	Registrar	Approved section list, validation report	Confirmation number	Write to official record
PromoteFromWaitlist	WaitlistMonitor	PersonalAssistant	Section ID, promoted student ID	Promotion notice	Update session state; notify student

- Design Phase

Agent Model:



Service Model:

Service	Inputs	Outputs	Pre-conditions	Post-conditions
ValidateSchedule (Scheduler)	Candidate section list, student schedule, transcript	Validation report (clear, or list of conflicts/unmet prerequisites)	Student profile and transcript exist	Report is available to the Registration Assistant Agent
RankCandidates (PreferenceAdvisor)	Validated schedule set, student preferences	Ranked schedule list	Scheduler has validated at least one candidate	Ranked list is available to PersonalAssistant
AssessWorkload (WorkloadAdvisor)	Validated schedule	Workload advisory (optional)	Schedule already validated as conflict- free	None; advisory only
QuerySeats (CourseProvider)	Course code, term	Section list with seat counts and prerequisites	Course exists in the department catalogue	None
RequestEnrollment (CourseProvider)	Student ID, section ID	Enrollment confirmation or waitlist position	Validation report is clear for this section	Seat count decremented, or waitlist queue updated
PromoteFromWaitlist (WaitlistMonitor)	Section ID	Promotion notice	A seat has freed and the waitlist is non-empty	Seat count decremented; waitlist queue updated
TrackDemandTrend (DemandForecaster)	Enrollment history, waitlist queue	Demand alert (optional)	None	None; advisory only

Service	Inputs	Outputs	Pre-conditions	Post-conditions
FinalizeRegistration (Registrar)	Approved section list, validation report	Official confirmation number	All sections in the list are enrolled or waitlisted and validation is clear	Official academic record updated
LogRegistrationEvent (ComplianceAuditor)	Finalized registration record	Audit log entry	Registrar has finalized the transaction	Audit log updated; record itself unchanged

Acquaintance Model:

The Registration Assistant Agent is the only agent type that communicates directly with the student, and every other agent conversation passes through it. It is the initiator in all but one case where the exception is the CourseAgent's WaitlistMonitor role, which initiates PromoteFromWaitlist back to the Registration Assistant Agent when a seat frees up.

- RegistrationAssistant speaks to the Scheduler Agent (requests schedule validation)
- RegistrationAssistant speaks to the CourseAgent (queries sections, submits enrollment/waitlist requests).
- RegistrationAssistant speaks to the RegistrarAgent (submits the approved schedule for finalization)
- CourseAgent speaks to RegistrationAssistant (WaitlistMonitor reports a waitlist promotion).

- **Agent Description:**

Agent - Registration Assistant Agent

Roles: PersonalAssistant (hard), PreferenceAdvisor (soft).

The Registration Assistant Agent is the student facing communicator of the system. Through the PersonalAssistant role it collects the student's desired courses and preferences through the browser interface, requests validation from the Schedule Agent, queries the relevant Course Agents for section options, and shows a consolidated set of choices back to the student. Once the student approves the selection, it submits the schedule to the Registrar Agent and relays the resulting confirmation or any later status change (including a WaitlistMonitor promotion) back to the student. Through the PreferenceAdvisor role it orders the validated candidates it receives so the student's likely best fit is shown first.

Agent: Scheduler Agent

Roles: Scheduler (hard), WorkloadAdvisor (soft)

The Scheduler Agent is responsible for validating any schedule proposed by the Registration Assistant Agent. Through the Scheduler role, it checks the proposed sections against the student's existing enrollments for time conflicts and against the student's transcript for missing prerequisites and returns a validation report that the Registration Assistant Agent must receive as "clear" before any enrollment request may proceed. Through the WorkloadAdvisor role, it attaches an optional advisory when an already valid schedule looks unusually heavy.

Agent: Course Agent

Roles: CourseProvider (hard), WaitlistMonitor (hard), DemandForecaster (soft)

Course Agent represents a single department's course and section data, one instance exists per department. Through the CourseProvider role, it responds to queries about seat availability, meeting times, and prerequisites, and processes enrolment and waitlist requests directed at its sections. Through the WaitlistMonitor role, it constantly watches its own waitlists and promotes the next eligible student when a seat frees up, notifying the Registration Assistant Agent directly rather than waiting to be asked. Through the DemandForecaster role, it tracks enrollment trends for its own department's use, with no effect on student facing behaviour.

Agent: Registrar Agent

Roles: Registrar (hard), ComplianceAuditor (soft)

The Registrar Agent holds the university's official registration record. Through its Registrar role, it accepts only schedules that have already been approved by the student and cleared by the Scheduler Agent, finalizes the enrollment and waitlist selections, and issues an official confirmation number back to the Registration Assistant Agent. Through the ComplianceAuditor role it records an audit log entry for every finalized transaction without altering the transaction itself.

Agent Internal Architecture

- Listener: Listens on the agent's message port for incoming requests from other agents or in the case of the Registration Assistant Agent, from the student's browser session.
- Interpreter: Parses incoming messages against the shared course/schedule message schema agreed upon by all agent types. So that a course code, section ID, or student ID means the same thing to every agent that reads it.
- Processor: Holds one internal handler per role the agent plays (e.g., the Course Agent's Processor holds a CourseProvider handler, a WaitlistMonitor handler, and a DemandForecaster handler), and uses the Interpreter's output (or internal timer for WaitlistMonitor) to get the correct handler and produce a result.
- Local store: The agent's own data such as a Course Agent's section table or the Scheduler Agent's view of a student's schedule.
- Responder: Formats the Processor's result back into the shared message schema and returns it to the requesting agent or initiates a new outgoing message.

Extensions and Limitations (Preliminary)

- The central coordinator model has most of the coordination in the Registration assistant Agent which the load itself could become a bottleneck. A future iteration might address by allowing Course Agents to communicate seat changes directly to a shared waitlist-monitoring service.